SCIPY INDIA 2026 | Technical supporting resource for a 30-minute talk

Engineering Reliable AI Inference

A Python-Based Case Study in Streaming, Error Handling, and Reproducibility

BitBoundPay (BBP) engineering team | Four contributors | Python, FastAPI, PostgreSQL, asyncio

Abstract

Applications that call hosted AI models look simple from the outside: send a prompt, receive tokens. In production, the call path is a distributed system with a slow, stateful, partially observable dependency in the middle. Responses arrive as long-lived streams; failures can occur before the first byte, mid-stream, or after the client has gone; and each provider reports the same underlying failure in a different shape. This document describes how the BBP team structured a Python service (FastAPI, asyncio, PostgreSQL) so that these behaviours are explicit, classified, and testable, and it separates what is **demonstrated** from what is **planned**. A companion public repository (`bbp-reliability-study`) contains a minimal reference implementation with mock providers; all results reported here were produced from it and can be regenerated with two commands. We present a provider-independent event vocabulary, a closed error taxonomy with a propagation rule for streams, a run lifecycle with exactly-once finalization, a deterministic test harness, one real production debugging case, and a limitation our own evaluation exposed.

1. The Engineering Problem

Why multi-provider AI applications are hard to test reliably. Four properties combine to defeat ordinary test strategies.

- **Non-determinism.** Model output varies between identical calls, so equality assertions on content are brittle. Reliability tests must assert on *behaviour* (event order, terminal state, error class), not on text.
- **Fault rarity.** Rate limits, mid-stream disconnects, and malformed chunks are infrequent against live providers, so a test suite that only uses live calls almost never exercises its own failure paths.
- **Heterogeneous contracts.** Providers differ in streaming framing, error payloads, status codes, usage reporting, and cancellation semantics. Without a boundary, these differences leak into business logic.
- **Cost and quota.** Live-provider tests consume paid quota and are themselves subject to the rate limits and outages they are meant to detect, making CI flaky and expensive.

Common failure categories. Table 1 lists the categories the system is designed around, with the signal a provider typically emits and the handling policy we adopted. Signals are described generically; exact codes vary by provider.

Category	Typical signal	Retry?	Handling policy
Authentication / authorization	401/403, invalid or expired key	No	Fail fast; surface a generic public message; never retried
Rate limit / quota	429, retry-after hint, quota-exhausted body	Bounded	Retried before first token; honours the hint, otherwise exponential backoff with seeded jitter
Invalid request / context too long	400, schema or token-limit error	No	Return actionable client error; do not forward provider text verbatim
Provider unavailable	5xx, overloaded response	Bounded	Retried before first token only; after that, reported to the client
Timeout / network	Connect or read timeout, reset	Bounded	Classified as TIMEOUT; retried before first token. Per-phase timeouts are a design target, not in the reference
Malformed or truncated stream	Bad JSON chunk, missing terminal event	No	Mark run failed; keep partial text; flag as incomplete
Client disconnect	Socket closed by consumer	N/A	Cancel upstream call; finalize as aborted, not failed

Table 1. Failure categories and handling policy.

2. Provider-Independent Execution

The central design rule is that **only provider adapters may know a provider's wire format**. Each adapter implements one small asynchronous interface and emits a closed set of normalized events. The orchestrator, conversation service, and API layer depend on that vocabulary and nothing else. Figure 1 shows the resulting request path.

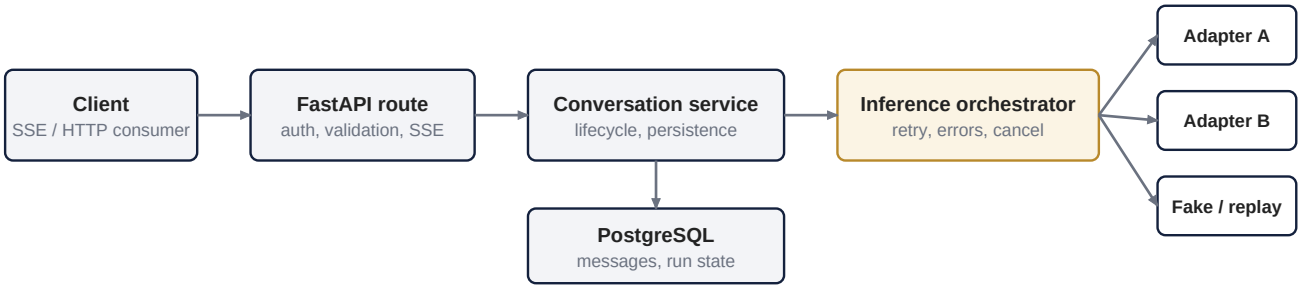

Fig. 1. Provider-independent request path. Adapters emit one normalized StreamEvent vocabulary; nothing above them sees provider wire formats.

The interface is deliberately narrow. Listings in this document are taken from, or simplified from, the public reference repository, not from BBP production code.

```
@dataclass(frozen=True)
class StreamEvent:
    kind: str                          # "delta" | "usage" | "done" | "error"
    text: Optional[str] = None         # for "delta"
    usage: Optional[dict] = None       # for "usage"
    error: Optional[InferenceError] = None

class ProviderAdapter(Protocol):
    name: str
    def stream(self, req: dict) -> AsyncIterator[StreamEvent]: ...
    # Contract: zero or more "delta", then exactly one "done" or "error".
    # An adapter translates provider exceptions into "error" events.
```

2.1 Request processing and streaming workflow

A single chat request follows seven steps. The ordering matters because it determines what is durable if the process stops at any point.

1. **Admit.** Authenticate and validate. Failures here are returned as ordinary HTTP errors because no stream has started.
2. **Persist intent.** Commit the user message and create a run record in state PENDING before any provider call.
3. **Open the stream.** Return a `text/event-stream` response; from this point the HTTP status is fixed at 200.
4. **Execute.** The orchestrator selects an adapter and iterates its event stream, applying the retry policy only until the first delta arrives.
5. **Forward and buffer.** Each delta is framed as an SSE event for the client and appended to an in-memory buffer for later persistence.
6. **Finalize.** On any terminal event, write the assistant message and terminal run state in one transaction.
7. **Release.** Close the upstream stream (the reference uses `contextlib.aclosing`), so a retry or abort never leaves a provider connection open.

Two consequences follow from step 3. First, **errors after the stream opens cannot change the HTTP status**, so they must travel in-band as a typed event (Section 3). Second, because cancellation can arrive at any await point, finalization must not depend on the happy path continuing (Section 4).

3. Error Classification and Propagation

Errors are represented by a closed taxonomy rather than by provider exceptions or free-form strings. Each error carries a stable machine-readable code, a retryability flag, and a user-safe message; raw provider detail is kept for logs only.

```
class ErrorClass(str, Enum):
    AUTH = "auth"; RATE_LIMIT = "rate_limit"; INVALID_REQUEST = "invalid_request"
    UNAVAILABLE = "unavailable"; TIMEOUT = "timeout"; PROTOCOL = "protocol"
    CANCELLED = "cancelled"; INTERNAL = "internal"

@dataclass(frozen=True)
class InferenceError:
    cls: ErrorClass
    retryable: bool
    retry_after_s: float | None
    public_message: str        # safe to show to end users
    detail: str                # logs only; never sent to the client
```

Propagation rule. Before the first byte, an error maps to an HTTP status through a single table. After the first byte, the same error is emitted as a final `error` SSE event carrying the class and public message, followed by stream close. Clients therefore need exactly one rule: *a stream that does not end with a done event has failed, and the last event says why.* The mapping lives in one module, which keeps the policy reviewable and lets tests enumerate it exhaustively.

Retry boundary. Retrying after tokens have been shown to the user risks duplicated or contradictory output, so automatic retry is limited to the window before the first delta. After that point, the system reports the failure and preserves the partial text rather than silently restarting.

4. Conversation Lifecycle and State Management

A conversation is a durable sequence of messages; a *run* is one attempt to produce the next assistant message. Separating the two lets a failed or aborted run leave the conversation consistent and resumable. Figure 2 shows the run state machine.

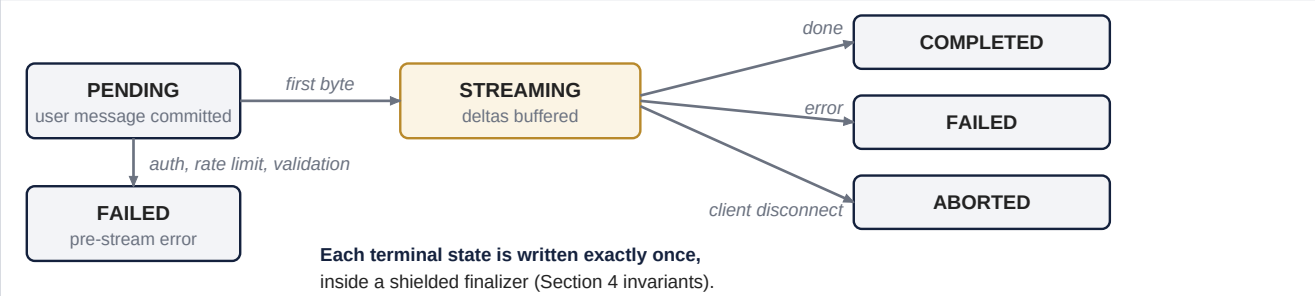

Fig. 2. Run lifecycle. Pre-stream failures move directly from PENDING to FAILED; stream outcomes are COMPLETED, FAILED, or ABORTED.

Invariants. The following properties are enforced by code structure and checked by tests in the reference:

- The user message is committed **before** any provider call, so a crash never loses the input.
- Each run reaches exactly one terminal state, and the assistant message is written at most once per run (guarded by a unique constraint on the run identifier).
- Finalization runs in a `finally` block through `asyncio.shield`, so cancellation of the request handler does not interrupt the write. Exactly-once is enforced by a guarded UPDATE (state must still be PENDING or STREAMING) and a UNIQUE constraint on (run, role).
- The public reference does not implement idempotency keys or resource reservation; they are outside its scope.

Why abort is a first-class state. A disconnecting client raises `asyncio.CancelledError` inside the streaming generator. Treating this as an error conflates user intent with system failure and, worse, tempts the handler to skip persistence. Modelling ABORTED separately keeps partial output, prevents false alarms in error-rate dashboards, and makes cancellation a tested path.

```
async def run_stream(store, adapter, req, run_id, **kw):
    buffer, terminal, aborted = [], None, False
    try:
        async for ev in execute(adapter, req, **kw):
            if ev.kind == "delta": buffer.append(ev.text)
            if ev.terminal: terminal = ev
            yield sse_frame(ev)
            if terminal: break
    except (asyncio.CancelledError, GeneratorExit):   # client disconnect
        aborted = True
        raise
    finally:   # success, error and disconnect all finalise here
        await asyncio.shield(_finalize(store, run_id, "".join(buffer), terminal, aborted))
```

5. Repeatable Testing and Reproducibility

The strategy follows from the problem statement: make faults cheap to produce and every run replayable. Table 2 states which layers exist in the public reference and which are planned; nothing marked planned is claimed as done.

Layer	Technique	Status
Orchestrator	Scripted fake provider; injected fake clock; seeded RNG for jitter	Implemented (public repo)
Stream invariants	Property-based tests (Hypothesis, 100 generated cases)	Implemented (public repo)
Lifecycle persistence	In-memory SQLite; disconnect and mid-stream failure tests	Implemented (public repo)
SSE framing	Direct assertions on emitted frames, including error-detail redaction	Implemented (public repo)
Adapter contract	Shared suite over real adapters using recorded payloads	Planned; not in public repo
PostgreSQL and ASGI end to end	Lifecycle tests against PostgreSQL; ASGI test client	Planned; not in public repo
Live smoke	Small quota-capped run against real providers, outside gating CI	Planned; not in public repo

Table 2. Test layers and their status.

Determinism controls. Time and randomness are injected: a fake clock records requested sleeps instead of sleeping, and jitter takes a seeded random source. A fault scenario is a list of steps (deltas, a fault, a terminal), and a multi-attempt scenario is a list of such lists, so each scenario is data in a version-controlled test file. A representative test from the repository:

```
def test_midstream_failure_keeps_partial_text():
    s = RunStore(); rid = s.create_run("c1", "hi")
    run(collect_frames(s, FakeAdapter([Delta("Hello"), Delta(" wor"),
                                       Fault(ErrorClass.UNAVAILABLE)]), rid))
    assert s.run(rid) == ("FAILED", "unavailable") and s.assistant_text(rid) == "Hello wor"
```

6. Reliability Evaluation: Results

All results below come from the public reference implementation with a mock provider (Python 3.12.3, pytest 9.1.1, Hypothesis 6.169.3, Linux x86_64, one machine). The suite has 25 tests; all 25 pass. Table 3 lists a representative subset, and Table 4 reports the measurements. Both regenerate with `pytest` and `python examples/evaluate.py`.

Test scenario	Asserted behaviour	Result
Provider fault before first token	Retried; stream completes; 2 provider calls, 1 sleep	PASS
Rate limit with retry-after = 2.0 s	Sleep log equals [2.0]	PASS
Rate limit after first delta	Not retried; final event is error; 1 provider call	PASS
Authentication failure	Fails fast; class auth; 1 provider call	PASS
Persistent unavailability, max 2 retries	3 calls, 2 sleeps, final event is error	PASS
Stream ends without terminal event	Final event is a protocol error	PASS
Mid-stream failure	Run FAILED; stored text is exactly the delivered "Hello wor"	PASS
Client disconnects after first frame	Run ABORTED, not FAILED; stored text "one"	PASS
Finalize called twice	Second call returns False; one assistant message	PASS
Randomly generated scripts (property)	One terminal event; one assistant message; stored text equals delivered text	PASS
Disconnect before first frame	Run remains PENDING (documents a known limitation)	PASS (documents behaviour)

Table 3. Representative tests from the public suite (25 of 25 pass).

Measurement	Method	Observed result
Replay determinism	5 scenarios x 1,000 repeats; compare frames, final state, stored text, sleep log	5,000 of 5,000 identical
Error classification	Inject each of 7 non-cancel error classes at first byte	7 of 7 mapped to the intended class
Abort-point sweep	3 scripts, client closes after 1 to 5 frames; plus task cancellation during a provider await	15 of 15 reached a terminal state with one assistant message and stored text equal to delivered text; cancellation case ABORTED with partial text kept
Abort before first frame	Same 3 scripts, close before any frame	**3 of 3 left the run PENDING** (limitation, below)

Measurement	Method	Observed result
Pipeline overhead to first frame (microseconds, p50 / p95 / p99)	3,000 paired runs: direct adapter vs full pipeline with SSE framing and SQLite writes	Direct 14.1 / 26.6 / 66.7; pipeline 51.3 / 100.3 / 142.1

Table 4. Measurements from `examples/evaluate.py`. Timings are machine-dependent.

Limitation found by our own evaluation. When a consumer closes the stream before receiving any frame, the generator body never starts, so its `finally` block never runs and the run stays PENDING. The reference documents this in a test; the intended mitigation is a reaper that marks stale PENDING runs ABORTED, which is not implemented. The overhead figures include an in-memory SQLite commit and a zero-latency fake provider, so they bound the abstraction cost only and say nothing about end-to-end latency or PostgreSQL.

Planned experiments (not yet run; no results claimed). (a) The abort-point sweep against PostgreSQL with the production service; (b) end-to-end time to first byte with real providers and network; (c) adapter contract tests over recorded real payloads; (d) a stale-PENDING reaper and its false-abort rate.

7. Engineering Lessons

Case study: lost conversation data on stream abort (BBP production).

Aspect	Description
Symptom	When a client aborted a streaming chat response, the conversation's content was not persisted.
Root cause	Persistence ran on the success path; on abort, database state was left uncommitted and discarded. A second defect on the same path: the abort handler used `asyncio` without importing it, raising NameError only when an abort occurred. Happy-path tests could not reach either defect.
Fix	Persist from a finalizer that runs on every exit and commits; add the missing import.
Verification	The abort path is covered in the public reference by the disconnect and mid-stream tests (Table 3) and the 15-case sweep (Table 4). The abort-before-first-frame case remains open.

Table 5. One debugging case, from symptom to verification.

- **Abort and error paths need their own tests.** Both defects were invisible to happy-path tests, which is why the sweep closes the stream at every frame boundary.
- **Order of side effects is part of the design.** In an earlier BBP version the provider was called before a usage quota was reserved; the fix is a fixed order of reserve, call, settle or release. This ordering is production behaviour and is not part of the public reference.
- **Process memory is not shared state.** An in-memory rate limiter was bypassed once the service ran several workers; limits and run state belong in shared storage.

8. Design Decisions, Alternatives, and Costs

Decision	Alternative not chosen	Cost accepted
Normalized event vocabulary at the adapter boundary	Pass provider payloads through to callers	Provider-specific features need explicit extension points
Closed error taxonomy with one mapping module	Propagate provider exceptions or strings	Unknown errors fall back to INTERNAL until classified
Retry only before the first token	Restart the call after partial output	Some recoverable mid-stream faults surface as failures
Run record separate from conversation	Write the assistant message directly	Extra table and state machine
Scripted fake providers as the main substrate	Live-provider tests in CI	Fidelity depends on how well the scripts model providers

Table 6. Principal design decisions. The alternatives listed are those considered for the reference design.

9. Reproducibility and Scope

- **Companion repository:** `bbp-reliability-study` (MIT licence): `docs/`, `src/`, `tests/`, `examples/`. Mock providers only; no credentials, endpoints, logs or production code.
- **Regenerate:** `pytest` reproduces Table 3; `python examples/evaluate.py` reproduces Table 4. Scenarios, clock and randomness are injected and seeded.
- **Not claimed:** production latency, production PostgreSQL results, or provider-specific behaviour. SQLite stands in for PostgreSQL in the reference; the exactly-once guarantee relies only on a guarded UPDATE and a UNIQUE constraint.
- **Talk outline (30 min):** failure taxonomy (5) | provider-independent execution and streaming (7) | error propagation and lifecycle (7) | live fault-injection demo (6) | results, limitation, lessons (5).